8회를 한 장으로:
설계 이유 총정리와 장애 대응

층마다 고른 이유 · 실무 점검 포인트 · 공통 환경 장애 진단과 복구

새로운 기능을 더하기보다 지금까지 만든 파이프라인의 설계 근거와 복구 절차를 묶어 점검합니다

오늘의 위치: 새 기능 없이 묶고 점검하기

1~7회차: 만들고 다듬기

CDC 수집과 Bronze 원문 적재
Silver 현재 상태 · Gold 마트
증분 갱신과 Iceberg 유지보수

1부: 설계 이유 총정리

층마다 무엇을 골랐는가
왜 그 방식을 선택했는가
실무에서 무엇을 볼 것인가

2부: 장애 대응과 마무리

공통 환경 실제 사고 타임라인
상류·하류 복구 순서와 검증
발표는 별도 날짜에 진행

정답 코드를 외우는 대신 요구사항에서 설계 선택과 운영 근거를 연결하는 것이 목표입니다

전체 흐름 한 장: 층마다 맡은 일

수집 · 원문 보존 (1~3회차)

상태 수렴 · 지표 · 운영 (4~8회차)

PostgreSQL
원천 트랜잭션
WAL · FULL

→

Debezium + Kafka
초기 스냅샷·변경
토픽별 보관

→

Flink → Bronze
토픽별 테이블
원문 append

→

Silver 현재 상태
LSN·offset 가드
is_deleted 보존

→

Silver core
dim · fct 3개
공통 정제·계약

→

Gold
주문·일별 마트
전체·증분 대조

→

조회 · 카탈로그
SQL·리니지
정리·복구

실시간 적재와 마트 갱신의 경계

Flink는 10초 체크포인트마다 Bronze 커밋, Silver·Gold는 배치(수동·Airflow 5분 주기)로 갱신

저장소 · 카탈로그 · 복구 볼륨

파일은 MinIO, 현재 참조는 JDBC Catalog, 복구 지점은 flink-state 볼륨

한 층에 모든 책임을 몰아넣지 않고 원문 보존·현재 상태·공통 모델·소비자 지표를 나눴습니다

핵심 질문 6개는 어느 층이 답하나

핵심 질문	답하는 층	핵심 장치 · 근거
무엇이 왜 바뀌었나 · 어디까지 왔나	원천 · Kafka · Bronze	c/u/d·before/after 원문 · topic·partition·offset
지금 유효한 상태와 버전은 무엇인가	Silver 현재 상태	숫자형 LSN·offset ROW_NUMBER + MERGE 가드
여러 마트가 같은 기준과 열 모양을 쓰는가	Silver core · 거버넌스	dim·fct 공통 모델 · contract · 데이터 테스트 15개
수정·삭제·재시도·장애 뒤에도 답이 맞는가	Gold · 유지보수 · 복구	영향 키·날짜 MERGE · 4대 정리 · 층별 재시작점

질문마다 답하는 층을 분리했기 때문에 문제가 생겼을 때 어느 층부터 조사할지 바로 좁힐 수 있습니다

수집(2회차): 왜 로그 기반 CDC인가

고른 것	왜 골랐나	실무에서 볼 것
WAL 논리 복제 (pgoutput)	원천 조회 부하 없이 중간 변경·물리 삭제 포착	slot 미소비 시 WAL 누적 · 디스크 감시
REPLICA IDENTITY FULL	UPDATE·DELETE의 before 행 전체 값 확인	WAL 증가 부담 · PK만 필요한 테이블은 DEFAULT
snapshot.mode = initial	최초 기존 행(op=r) 읽은 뒤 저장된 offset부터 이어받기	대용량 초기 스냅샷 시간 · 스키마 변경 시 재수집
op=d 보존 · tombstone 제외	삭제 직전 값은 남기고 value=null 레코드는 필터	Kafka log compaction 쓰면 tombstone 처리 구분

원천에 조회 부하를 주지 않고 삭제와 중간 변경까지 잡으려고 로그 기반 CDC를 골랐습니다

Bronze(3회차): 고치지 않고 쌓는다

고른 것	왜 골랐나	실무에서 볼 것
raw payload + 위치 정보 append	해석 규칙이 바뀌거나 버그가 나도 원문에서 재처리	원문 보관 비용 · 개인정보(PII) 마스킹·접근 통제
토픽마다 테이블 1:1 분리	하류 Silver가 필요한 원천 테이블만 읽도록 격리	테이블 수 증가 시 카탈로그·작은 파일 관리
체크포인트 완료 후 Iceberg 커밋	읽은 offset과 파일 참조를 맞춰 중복 커밋 방지	10초 주기의 빈 커밋·메타데이터 누적 (7회차)
group-offsets + earliest	복구 상태 없을 때 커밋된 위치나 처음부터 재개	Kafka 보관 기간 만료 전 복구 · 중복 유입 대비

Bronze는 파싱이나 중복 제거를 하지 않고 재처리의 출발점이 될 원문과 전달 위치만 남깁니다

Silver 현재 상태(4회차): 버전으로 고른다

고른 것	왜 골랐나	실무에서 볼 것
ROW_NUMBER (LSN·offset 순)	같은 입력 배치 안에서 PK별 최신 후보 1건 선택	단일 원천·파티션 전제 · 다중 파티션 시 키 라우팅
WHEN MATCHED 버전 가드	늦게 도착한 과거 이벤트가 현재 상태를 덮지 못하게 차단	ON 절에 넣으면 NOT MATCHED로 빠져 중복 INSERT
삭제를 후보 포함 + is_deleted	삭제를 먼저 빼면 옛 UPDATE가 부활하는 문제 방지	장기 보존 시 삭제 표시 행 정리 주기 결정
무효 PK·LSN 사전 검증 중단	Spark 기본 CAST의 NULL 변환과 중복 적재 차단	실무에서는 격리 테이블(dead-letter) 분리 검토

배치 안에서는 ROW_NUMBER로, 기존 테이블과는 MERGE 버전 가드로 비교해 역전을 막습니다

Silver core·Gold(5회차): 규칙은 한 곳에

고른 것	왜	실무에서 볼 것
Silver 2단 (현재 상태 → core)	삭제 제외·주문일(서울)·주문 키 공유	여러 마트가 같은 기준을 쓰는가
지표 정의·선집계 조인	성공 건만 집계 · 1:N fan-out 방지	회계 매출과 차이 · 상태 값 추가
dbt table 전체 교체 (5개)	CREATE OR REPLACE로 단순·멱등	30분 주기 안 완료 · 스냅샷 누적
모델 계약 5개 + 테스트 15개	생성 전 스키마 차단 · 생성 후 값 검사	실패 시 상류만 교체된 시점 불일치

공통 규칙은 Silver core에, 소비자 지표는 Gold에 두고 계약과 테스트로 지킵니다

증분(6회차): 바뀐 것만, 그래도 같은 답

고른 것	왜	실무에서 볼 것
Silver 현재 상태 changelog	바뀐 키·삭제 보존, carry-over 제거	전체 시간·변경 비율·MERGE 비용
영향 키 전파 후 전체 재합산	환불→결제→주문 추적 · 부호 꼬임 방지	삭제 거르기 전 변경 뷰에서 키 포착
이전·새 날짜 + 0건 그룹 삭제	주문일 정정 시 옛 날짜 유령 합계 제거	그룹 이동이 일어나는 모든 집계 열
plan 선저장 · progress 후갱신	두 마트 사이 실패 뒤 재시도 멱등성	일·주 단위 전체 재계산 대조(EXCEPT)

변경 감지로 범위만 좁히고, 선택된 키와 날짜는 전체 계산과 똑같이 다시 맞춥니다

유지보수(7회차): 쌓인 것을 안전하게 치운다

고른 것	왜	실무에서 볼 것
compact → expire 순서	합친 뒤 만료해야 옛 작은 파일 삭제	압축 직후엔 용량이 일시 증가함
orphans → manifests 정리	추적 밖 파일 삭제 · 매니페스트 병합	24시간 안전장치 · prefix_listing
metadata 자동 삭제 속성	커밋 때 한도 밖 metadata.json 삭제	기본값이 false라 명시 설정 필요
progress · Flink 스냅샷 보존	증분 시작점·체크포인트 복구선 보호	만료 사고 시 전체 재계산 비상 경로

합치기(compact)와 실제로 지우기(expire·orphans)를 구분하고 하류 기준선 이전까지만 지웁니다

거버넌스: 약속(계약)과 지도(리니지)

열 모양: 모델 계약

_*__models.yml에 타입 명시
테이블 교체 전 사전 검증
미합의 스키마 변경 차단

데이터 내용: 테스트 15개

키 유일성·결측·참조 관계
singular로 정산 차액 검증
OpenMetadata에 결과 수집

파급 지도: 컬럼 리니지

source·ref 컴파일 SQL 해석
amount → paid · net_paid 추적
영향 마트·열을 한눈에 확인

코드에 적은 설명·계약·테스트가 카탈로그 화면의 리니지와 품질 지표가 됩니다

Airflow: 언제·몇 번은 스케줄러, 다시 돌려도 안전은 작업

현재 저장소 DAG (코드)

5분 주기 · max_active_runs=1
catchup=False · retries=0
runner API로 Silver+dbt 호출

스케줄러가 맡는 일

언제 실행할지 (주기·타임아웃)
동시 실행 차단 (겹침 방지)
실패 시 몇 번 다시 부를지

작업 안이 맡는 일

버전 MERGE · 테이블 교체
plan 먼저, progress 나중
다시 불러도 같은 결과

```
# airflow/dags/lakehouse_refresh.py (발췌)
with DAG('lakehouse_refresh', start_date=datetime(2026,1,1,tzinfo=timezone.utc),
        schedule='*/5 * * * *', catchup=False, max_active_runs=1,
        is_paused_upon_creation=True, default_args={'retries':0},
        dagrun_timeout=timedelta(minutes=25), tags=['cdc','iceberg','dbt']) as dag:
    PythonOperator(task_id='silver_then_dbt_build', python_callable=refresh)
```

Airflow는 작업을 다시 호출할 뿐이고, 재시도해도 숫자가 안 틀리는 건 작업의 멱등성 설계가 지킵니다

설계 결정 한 장: 고른 것 · 대안 · 감수한 비용

고른 것 (우리 스터디)	대안	감수한 비용 · 전환 조건
Bronze 전체 재조회 → Silver	Bronze 스냅샷 증분 읽기	Bronze 커지면 스캔 증가 → 구간 고정 전환
dbt table 전체 교체 (5회차)	dbt incremental · 커스텀 증분(6회차)	전체 재계산 비용 vs 삭제·날짜·plan 복잡도
Copy-on-Write (COW) MERGE	Merge-on-Read (MOR)	쓰기 때 파일 재작성·carry-over vs 읽기 병합
Flink 스트리밍 Bronze + 배치 하류	전 구간 스트리밍 또는 전 구간 배치	상류 상시 커밋·메타데이터 누적 관리 필요

모든 선택에는 대안과 비용이 있으며, 데이터 크기와 신선도 요구가 바뀔 때 전환합니다

실무로 옮기면 달라지는 것

규모: 파티션과 압축 주기

일·월 단위 숨은 파티셔닝
쓰기 빈도에 맞춘 compact 주기
파일 크기·정렬(sort) 측정 후 도입

여러 팀: 계약과 소유권

Silver core는 공통 플랫폼 팀
Gold 마트는 도메인 팀이 소유
PR·CI 단계에서 계약 검증

비용·보존: 상류와 하류 맞춤

PG WAL · Kafka 7일 보관 한계
Iceberg 스냅샷 보존 기간
최대 장애 복구 시간보다 길게

혼자 돌리는 실습 환경에서 팀 단위 운영 환경으로 갈 때는 규모·소유·보존 기간을 먼저 맞춥니다

사고 타임라인: 조용히 쌓이다 한꺼번에 멈췄다

시각 (KST)	무슨 일이 일어났나	남은 흔적과 영향
11/1 16:51 ~ 3.4일	약 100초마다 빈 커밋 누적	테이블당 snapshot 2,952개 · 3.6GB
11/5 01:39 ~ 02:32	디스크 만원 → Flink·Kafka 중단	chk-29615 · Connect 로그 2.1GB
11/5 19:10 · 11/8	Spark·OM 장애 → 응급 7.7GB 확보	원인 미제거로 디스크 94% 재도달
11/13 16:21 ~ 16:36	메타데이터 정리 · 상류부터 복구	warehouse 13.75 → 3.08GB · 검증 통과

스터디 공통 환경 실측(11/1~11/13, KST) · 응급 공간 확보만으로는 며칠 뒤 같은 장애가 다시 옵니다

진단: 증상에서 원인까지 거슬러 가기

겉으로 보인 증상

Explorer Spark 연결 실패
OM 인덱스 읽기 전용 잠금
Connect 재연결 로그 2.1GB

→

용량과 프로세스 확인

디스크 94% (잔여 2.6GB)
Kafka·Connect 실제론 exited
Flink 실행 작업 0개

→

상류·저장소 대조

PG 미전달 WAL 760B
warehouse 13.75GB 중
Bronze metadata 각 3.6GB

```
# 스터디 공통 환경 진단 요약 (11/13 16:21 KST)
# 1) 디스크: 94% 사용 (남은 공간 2.6GB) / warehouse 13.75GB
#    bronze.orders/payments/refunds metadata: 3.60 + 3.61 + 3.60GB (데이터 각 <1MB)
# 2) 컨테이너: Kafka exited(1), Connect exited, Flink running jobs = 0
# 3) Flink 상태 파일: job_id만 있고 savepoint 없음
#    -> flink_state.py prepare: "Previous job has no saved stop point. Recover explicitly"
# 4) PostgreSQL slot 'cdc_lab': active=false, 미전달 WAL 760B (원천 27·27·7행)
```

실측(11/13) · 컨테이너 목록 표시만 믿지 말고 디스크·실제 프로세스·슬롯·체크포인트를 함께 봅니다

복구 순서와 그 이유

① 멈춤 확인 · 스냅샷 만료

쓰기 중단·행 수·해시 기록
속성 3개 설정 · retain_last=1
고아 삭제는 복원 뒤로 보류

② 상류부터 차례로 기동

Kafka 기동 · 지연 0 확인
Connect 기동 · slot 활성화
Flink chk-29615 명시 복원

③ 종단 검증 뒤 고아 삭제

verify.sh 6케이스 · dbt 검증
통과 후 24h 이전 고아 삭제
복원 전 삭제 위험 차단

안전한 메타데이터 만료로 숨통을 틔우고, 상류부터 복원해 검증한 뒤에 고아 파일을 지웁니다

복구 실측: 전후 숫자

스냅샷 · 메타데이터

스냅샷 2,952 → 1개
metadata.json 2,953 → 21개
파일 크기 2.4MB → 12KB

창고 용량 · 데이터 무결성

테이블당 3.60GB → 37~46MB
warehouse 13.75 → 3.08GB
행 수 27·28·63행 해시 동일

삭제 직후 디스크 확인

약 10GB 삭제 직후엔 92%
저장소 휴지통이 비워진 뒤
약 5분 만에 66%

```
# 스터디 공통 환경 복구 전후 실측 (11/13 16:21 vs 16:36 KST)
# 항목                    | 복구 전              | 복구 후
# ------------------------+----------------------+--------------------------
# Bronze snapshots / 로그   | 2,952개 / 101개      | 1개 / 21개 (manifest 2,951개 삭제)
# metadata.json 수 / 크기   | 2,953개 / 최신 2.4MB | 21개 / 최신 12KB (고아 2,863~2,865개 삭제)
# Bronze 테이블별 metadata  | 3.60 · 3.61 · 3.60GB | 46 · 46 · 37MB (행·해시 27·28·63 동일)
# warehouse 전체 / 디스크   | 13.75GB / 94%        | 3.08GB / 66% (삭제 직후 92% → 약 5분 뒤)
# Kafka 지연 / Flink 복원   | exited / 작업 0개     | 지연 0 (27·28·80) / restored 1, failed 0
```

실측(11/13) · 데이터 파일 삭제 0건으로 행과 해시를 지킨 채 창고 용량을 13.75GB에서 3.08GB로 줄였습니다

복구 검증: 컨테이너가 떴다고 끝이 아니다

검증 단계 (11/13 16:32)	실측 결과	확인한 의미
상류 오프셋 · 소비 지연	오프셋 27·28·80 · 지연 0	Bronze 27·28·63행(tombstone 17 제외) 일치
verify.sh 6개 시나리오	100k→80k→80k→70k→100k→100k	추가·환불·중복·수정·삭제·역전 방어 일치
dbt build 품질·계약	모델 5개 + 테스트 15개 PASS=20	Silver core · Gold 계약과 정산 일치
종단 반영 소요 시간	Bronze 4.9~12.8초 · Gold 16.2~25.1초	원천 변경이 끝까지 흐르는지 실측 증명

실측(11/13) · 원천에 새 변경을 넣어 Gold 숫자와 테스트 20개까지 통과해야 복구 완료입니다

재발 방지: 속성 · 주기 · 감시

구분	적용·설계 항목	실측 적용값 · 운영 기준
테이블 속성 (실측 적용)	metadata 자동 삭제 · 빈 커밋 완화	delete=true · max=20 · empty=360
정기 정리 (설계)	snapshot 만료 · 고아 파일 삭제	최신 snapshot 보존 · 24h 이전만
상시 감시 (설계)	디스크 · snapshot 수 · 마지막 커밋	디스크 80% 경고 · WAL · 실제 상태
유휴 시간 운영 (코드)	안 쓰는 동안 계획된 종료	stop.sh로 savepoint 남기고 중지

실측(11/13 속성 적용) + 설계 · 생성 템플릿 반영과 정기 정리·감시까지 묶어야 재발을 막습니다

층별 재처리 시작점: 어디서 멈추면 어디서 다시

멈춘 곳	다시 시작하는 지점	한계·주의
수집 (Debezium · Kafka)	replication slot LSN · Connect 오프셋	멈춘 동안 원천 WAL 누적 · 디스크 주의
Bronze 적재 (Flink)	checkpoint · savepoint 소스 오프셋	Kafka 보관 기간(7일) 안에만 재개
Silver (현재 상태)	Bronze 전체 재조회 · 버전 MERGE	이력 커지면 지연 · 증분은 6회차 방식
Gold (dbt · 증분)	table 재실행 · 증분은 progress와 plan	progress 만료 시 전체 재계산 후 재설정

여드레 동안 멈췄어도 유실이 없었던 이유는 층마다 어디까지 처리했는지 기록하고 버전으로 고르기 때문입니다

활동: 내 프로젝트 장애 한 줄 쓰기

항목	공통 환경 사고 예시 (11/5~11/13)	내 프로젝트에 대입할 질문
① 증상과 관측	Spark 세션 실패 · Connect 로그 폭주	어느 화면·로그에서 처음 이상을 보나
② 원인과 멈춘 층	Bronze 메타데이터 3.6GB×3 → 디스크 만원	수집·Bronze·Silver·Gold 중 어디가 멈췄나
③ 재시작 지점	expire(retain 1) → Kafka → chk-29615 복원	slot·checkpoint·전체 재조회·progress 중 무엇인가
④ 복구 검증 기준	verify.sh 6개 케이스 · dbt PASS=20	단순 기동 말고 어떤 숫자·쿼리로 확인할까

수업 중 별도 제출은 없으며, 내 도메인 테이블 하나를 골라 네 칸을 채우고 발표 자료의 운영 절에 활용합니다

최종 운영 점검표

점검 영역	통과 기준 (labs/README.md 8회차)	대표 확인 방법
문서만으로 재현	다른 사람이 시작·조회·중지 가능	README 순서대로 bootstrap · SQL · stop
정합성 · 품질 검증	기대값 일치 · 계약과 테스트 통과	verify.sh · dbt build · EXCEPT 대조
장애 복구 · 재시도	중단 중 변경도 복구 후 마트 반영	recovery.sh · 진행 위치 보존 확인
유지보수 · 안전 종료	메타데이터·작은 파일 정리 · 데이터 보존	속성·expire 순서 · down -v 금지(stop.sh)

내 컴퓨터에서만 도는 코드가 아니라, 다른 사람이 문서만 보고 켜고 확인하고 안전하게 끌 수 있어야 완주입니다

8회를 마치며: 정답보다 설계 방법

① 층마다 분명한 책임

Bronze는 고치지 않고 보존
Silver는 버전으로 현재 상태
Gold는 약속된 지표와 계약

② 운영까지가 설계

증분은 전체 계산과 대조
커밋 뒤엔 만료와 고아 정리
장애 땐 상류부터 순서대로

③ 다음 단계: 발표 안내

8회차 이후 별도 날짜 진행
형식·기한은 별도 공지
설계 판단·문제 해결 공유

도구 사용법을 넘어 왜 이렇게 골랐고 어떤 비용을 감수했는지 설명할 수 있다면 충분합니다